Assignment 1 COMP5318

Group 149: 520510542, 510458096, 520131808, 550749464

September 2025

1 Introduction

In this study, we investigate the performance of a range of machine learning algorithms applied to the Rice dataset, which contains 1,400 samples from two rice varieties—*Cammeo* and *Osmancik*. Each sample is described by seven numerical attributes, and the goal is to develop classifiers capable of distinguishing between these varieties with high accuracy and reliability.

To ensure a fair and rigorous evaluation, multiple algorithms are considered, ranging from simple linear models such as Logistic Regression to probabilistic approaches like Naive Bayes, tree-based methods including Decision Trees and Random Forests, and ensemble techniques such as Bagging, AdaBoost, and Gradient Boosting. Instance-based learning with k-Nearest Neighbors is also explored. These algorithms represent a broad spectrum of machine learning paradigms, allowing for meaningful comparison of their effectiveness on the task.

The methodology emphasizes robust validation and systematic hyperparameter optimization. Models are trained and tested using 10-fold stratified cross-validation to preserve class balance and reduce variance in performance estimates. For models sensitive to parameter choices, such as Random Forests and k-Nearest Neighbors, grid search is employed to identify optimal hyperparameter configurations. Performance is reported in terms of accuracy, while additional metrics such as F1 scores and confusion matrices provide deeper insight into class-level behavior and model generalization.

By combining strong evaluation methodology with a diverse set of classifiers, this work aims not only to identify the best-performing model for rice variety classification but also to provide a clear comparative understanding of the strengths and limitations of each approach.

2 Methods

2.1 Evaluation

All models are evaluated using 10-fold stratified cross-validation. In this approach, the dataset is divided into 10 equal folds while preserving the original class distribution within each fold. For every iteration, a model is trained on 9 folds and tested on the remaining fold. This process is repeated 10 times, each fold serving as a test set once. The overall accuracy score is then averaged across the 10 iterations to provide a more robust estimate of the performance of the models.
In conjunction, the random_state for each model is set to 0 to ensure reproducibility. Setting the random_state to 0 sets the seed for the random number generator, meaning that the model structure, splits and performance metrics are the same each time, and allowing for fair comparisons between algorithms.

2.2 Parameter tuning

Grid search is a parameter tuning technique that finds the best hyperparameters for a model. Several possible values are picked for each hyperparameter, and a model is then trained for every possible combination, finally the set of hyper parameters that gives the best performance is chosen. We utilise parameter tuning with the K-Nearest Neighbour Classifier and the Random Forest Classifier

2.3 Preprocessing procedure

The data preprocessing procedure involves three implementation steps and one confirmation step. To begin with, we fill the missing attribute values with the mean value of the column using sklearn.impute.SimpleImputer. This is done to ensure that no rows with missing values are dropped and thus, we can obtain more accurate and less biased results. Secondly, we normalise the data using a min-max scaler of [0,1) with sklearn.preprocessing.MinMaxScaler. This is to ensure that large valued data (e.g perimiter, Major_Axis_Length, Area, Minor_Axis_Length, and Covex_Area) doesn't dominate smaller valued data (e.g Eccentricity, and Extent), as that would produce more biased results. Finally, we then change the class values, with class1 becoming 0 and class2 becoming 1. This is to ensure that we have a clean binary label format for all classifiers, allowing all classifiers to cleanly process the data without worry. To ensure that the data has been properly cleaned, we print out the first 10 rows in a formatted output.

2.4 Logistic regression

Intuition

Logistic regression is a linear classification method that models the probability of an instance belonging to a particular class using a sigmoid function. The

output of the sigmoid represents the predicted probability for each class.

Loss function

In scikit-learn, logistic regression uses log loss (cross entropy loss) and applies L2 regularization by default.

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\Big] + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$

Technical details

- **max_iter = 10000:** This argument limits the maximum number of iterations the solver can run. It also increases accuracy and guarantees convergance.
- **random_state = 0:** This option sets the seed for the random number generator so results are reproducible.

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3

2.5 Naive Bayes

Intuition

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It makes a simplifying assumption that all features are conditionally independent given the class label. Naive Bayes works by computing the probability of each class using the features of a data point and then assigning the class with the highest probability.

Technical details

Naive Bayes does not use a loss function, it uses probability estimates from the training data. Since the features in our dataset are entirely numerical, we use Gaussian Naive Bayes which estimates the probability from the mean and variance of each feature within each class.

$$P(x \mid y) = \frac{1}{\sqrt{2\pi\sigma_y^2}}\, e^{-\frac{(x-\mu_y)^2}{2\sigma_y^2}}$$

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3

2.6 Decision Tree

Intuition

A decision tree is a classifier that sorts data into subgroups by asking a series of questions about a datapoint's features. Each internal node represents a feature-based question, each edge corresponds to a decision, and each leaf node represents a predicted class label. If we have a datapoint to be predicted, we pass it through the decision tree and the leaf node it ends up on will be our prediction.

Technical details

Like Naive Bayes, decision trees do not rely on a loss function. Instead, during tree construction, the algorithm evaluates how "good" a split is by calculating its entropy, or Gini impurity.

In scikit-learn, the DecisionTreeClassifier selects the best feature to split on at each node based on a splitting criterion. By default, this criterion is the Gini impurity.

$$Gini(E) = 1 - \sum_{j=1}^{e} p_j^2$$

The Gini impurity measures how mixed the classes are at a node in a decision tree. It is calculated as above, where p_j is the proportion of class j. A value of 0 means the node is pure while higher values mean the node contains a mix of classes.

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3

2.7 Bagging, ada boosting and gradient boosting

Intuition

These three methods Bagging, AdaBoost, and Gradient Boosting are all ensemble techniques using decision trees. An ensemble consists of multiple decision trees trained for the same prediction task. The final prediction of the ensemble is obtained by combining the predictions of all trees, typically through majority voting for classification.

- **Bagging (Bootstrap Aggregating):** This method trains multiple decision trees on random resamples (with replacement) of the original dataset. Each resample is used to train a separate tree. Because some data points appear multiple times in a resample, the corresponding trees will perform better on those points in general.

- **AdaBoost:** AdaBoost trains decision trees sequentially, where each tree tries to correct the mistakes of the previous ones. After each tree is trained, the algorithm increases the weight of the misclassified data points, so the next tree focuses on predicting the previously mislabeled datapoints.
- **Gradient Boosting:** Gradient Boosting also trains decision trees sequentially, but instead of weighting misclassified points like AdaBoost, each new tree is trained to predict the residual errors (the difference between the actual and predicted values) of the previous trees.

Technical details

- **n_estimators=50:** this argument decide the number of trees in the ensemble (This only applies all three methods)
- **max_samples=100:** this argument decides the number of samples drawn from the original dataset to train each tree. (This only applies to bagging)
- **max_depth=5:** this argument decides the max depth for each tree. (This only applies to bagging and adaboost)
- **learning_rate=0.5:**
  - **For adaboost:** This is the weight assigned to each classifier during every boosting step. A higher learning rate makes each classifier contribute more.
  - **For gradient boosting:** The learning rate scales down the influence of each individual tree by this factor

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3

2.8 K Nearest Neighbour

Intuition

The KNN algorithm is an algorithm that predicts the class of a data point by looking at the K closest points in the training data, and taking the majority vote to predict its class.

Technical details

There are two hyper parameter we used grid search on with this model:

- **n_neighbours:** This specifies number of neighbours
- **p:** This is the power parameter.

- **When p=1:** the distance between points is calculated by using manhattan distance.
- **When p=2:** the distance between points is calculated by euclidean distance.

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3. Parameter tuning was performed via grid search as described in Section 2.2.

2.9 Random Forest

Intuition

The random forest is a classification method that builds an ensemble of decision trees by training each tree on a random subset of the data (bagging) and using a random subset of features at each split, reducing correlation between trees and improving overall performance.

Technical details

There are two hyper parameter we used grid search on with this model:

- **n_estimators:** which as described above is the number of decision trees in the ensemble.
- **max_leaf_nodes:** which specifies the max number of leave nodes in the ensemble.

Preprocessing and Evaluation

Evaluation and preprocessing follow the global procedures described in Sections 2.1 and 2.3. Parameter tuning was performed via grid search as described in Section 2.2.

3 Experiments

Preprocessing

We are going to use the Rice dataset which has 1400 values of Rice data belonging to two varieties either - "cammeo" and "osmancik". There are 7 attributes for these rice categories, which we will be using to build a Rice classifier which would be capable of classifying the test data into one of these categories based on its attribute values.

Firstly, we preprocess the dataset to make it suitable for training our classifiers. We perform the following pre-processing steps:

- The values in attribute "classes" which originally have the value "class1" and "class2" are changed into labels 0 and 1 respectively.
- We use SimpleImputer to replace the nan values with the mean values of each column or attribute.
- We normalise each attribute to lie in the range of [0,1] by using MinMAxScaler function for each column of the rice dataset.

Environment

All of the following experiments were conducted using Python 3.12.11 and the scikit-learn library 1.6.1 for implementing the classifiers. Numpy version 2.02 and Pandas version 2.2.2 were both used for dataset cleaning. Experiments were conducted on Google Colab with an Intel(R) Xeon(R) CPU @ 2.20GHz (1 physical core, 2 logical threads, 56 MB cache) and approximately 13 GB of RAM.

Part 1: Cross-validation without parameter tuning

We report the results for all of the different algorithms in Table 1. We report the accuracy as well as mention the fixed hyper-parameters used while implementing the different algorithms. We see from the results that Bagging ensemble for Decision Trees reports the highest accuracy. Furthermore, in Figure 1 we see the trends for all of the algorithms across the different cross validation sets. Along with giving us the best test scores, Bagging Ensemble for Decision trees has consistent results across the 10 cross-validation sets. AdaBoost has the most centered box plot with least variance across 10 cross-validation runs which solidifies the algorithm's performance. Decision trees without any ensemble perform the worst with the most variable performance across cross-validation sets as well as the lowest accuracy. Each algorithm has at least one outlier apart from naive decision trees. From the boxplot Figure 1, we can deduce that along with high accuracy, algorithms like Bagging, Ada Boost, Gradient Boosting give us more consistent performance compared to Logistic Regression, Naive Bayes and Naive Decision Tress for the different cross-validation sets.w

Table 1: Classifier performance on the Rice dataset using 10-fold Stratified Cross-Validation

Algorithm	**Hyperparameters**	**Accuracy**
Logistic Regression	—	0.9386
Naive Bayes	—	0.9264
Decision Tree	max_depth = None (default)	0.9057
Bagging (Decision Tree)	$n_estimators = 50$, max_samples = 100, max_depth = 5	0.9421
AdaBoost (Decision Tree)	$n_estimators = 50$, learning_rate = 0.5, max_depth = 5	0.9329
Gradient Boosting	$n_estimators = 50$, learning_rate = 0.5	0.9300

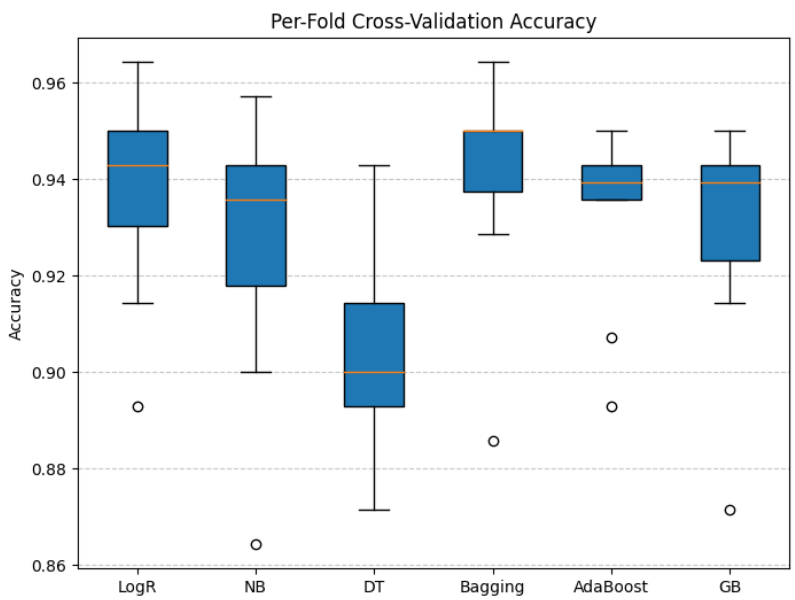

Figure 1: Cross Validation Accuracy for different algorithms across 10 runs.

Part 2: Cross-validation for parameter tuning

For the other two classifiers - K-Nearest Neighbor and Random Forest, we will find the best parameters using grid search with 10-fold stratified cross-validation as well as report different metrics on the cross-validation as well as the test sets. With the permission of our tutor, we change the input parameters for both classifiers from X and Y to X_train, X_test, y_train, and y_test, thus allowing us to pass through the training and test data.

We begin by reporting the best hyper-parameters and the corresponding cross -validation and test accuracy in Table 2. The accuracy metrics are very comparable for both of the classifiers with Random Forest performing slightly better. The close CV vs Test accuracy shows good generalization and no major overfitting.

We also report the F1 scores which are the harmonic mean of precision and recall for Random Forest in Table 3. Macro F1 treats all classes equally as it takes average of per-class F1 and Weighted F1 weights each class's F1 by its number of instances. The Random Forest achieved a macro-average F1 of 0.9326 and a weighted F1 of 0.9341. The closeness of these values indicates that performance is consistent across all classes, and that the dataset is well balanced. While accuracy gives an overall measure, the F1 scores reveal that Random Forest is not biased towards majority classes and maintains strong recall for minority classes as well.

Table 2: Hyperparameter tuning results for KNN and Random Forest on the Rice dataset

Algorithm	**Best Hyperparameters**	**CV Accuracy**	**Test Accuracy**
KNN	$k = 5$, $p = 1$	0.9371	0.9257
Random Forest	$n_estimators = 10$, max_leaf_nodes=12	0.9400	0.9343

Table 3: Additional performance metrics for Random Forest on the Rice dataset

Metric	**Value**
Macro Average F1	0.9326
Weighted Average F1	0.9341

In Figure 2 and Figure 3, we show how different hyper-parameters influence the algorithm's performance.

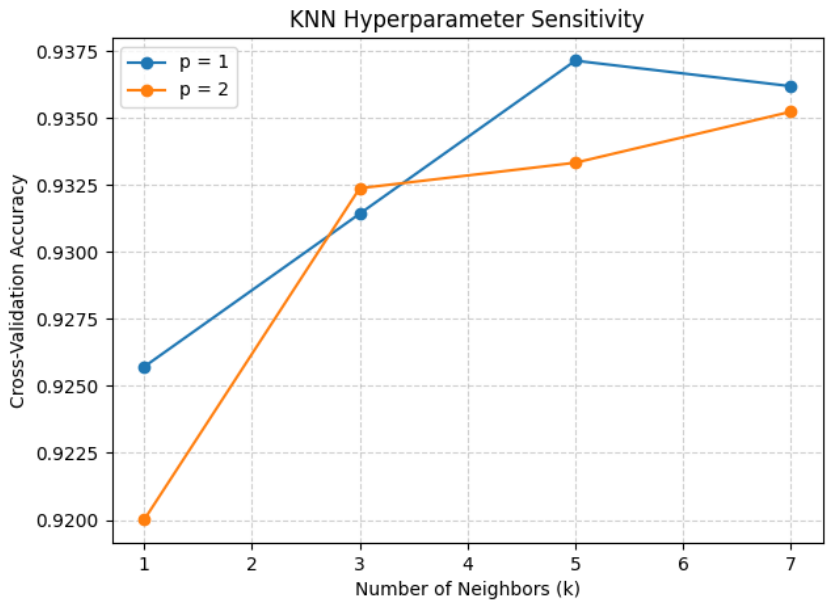

Figure 2: KNN algorithm Hyperparameter Sensitivity

K-Nearest Neighbors Sensitivity

The k-NN hyperparameter analysis demonstrates clear sensitivity to both neighborhood size and distance metric selection. Manhattan distance (p=1) consistently outperforms Euclidean distance (p=2) across all k values, with optimal performance achieved at k=5 (93.7% accuracy).

Key Findings:

- **Distance Metric Impact:** Manhattan distance shows superior performance, likely due to reduced sensitivity to irrelevant features in high-dimensional spaces.
- **Optimal Neighborhood Size:** k=5 provides the best bias-variance trade-off for Manhattan distance.
- **Performance Range:** Manhattan distance achieves 92.55%-93.7% accuracy vs. Euclidean's 92.0%-93.5%
- **Optimization Stability:** Clear inverted-U curve for Manhattan distance enables reliable parameter selection.

The Euclidean distance exhibits monotonic improvement with increasing k, suggesting the optimal value might lie beyond k=7, while Manhattan distance

shows a well-defined optimum, facilitating more straightforward hyperparameter tuning.

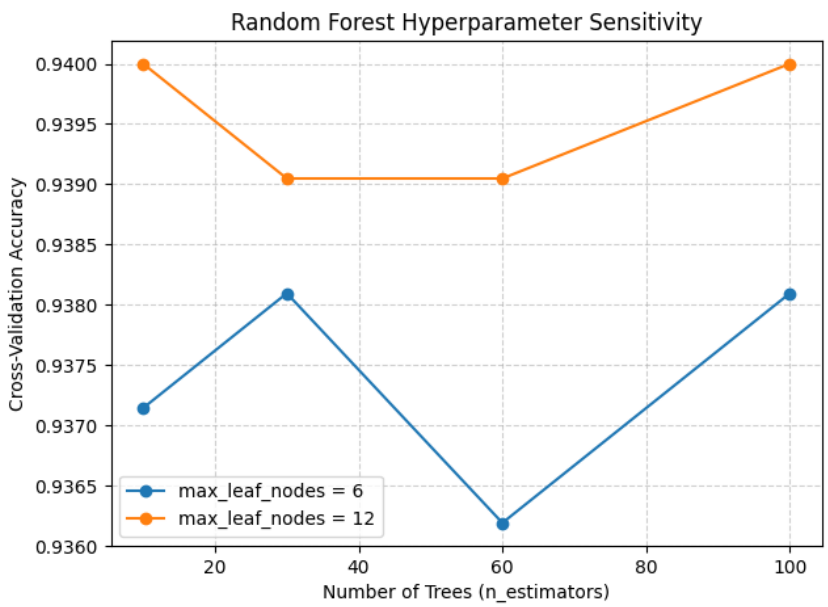

Figure 3: Random Forest Hyperparameter Sensitivity

Random Forest Sensitivity

Random Forest demonstrates more complex hyperparameter interactions, with performance varying significantly based on both ensemble size (n_estimators) and tree complexity (max_leaf_nodes).

Key Findings:

- **Tree Complexity Advantage:** Higher complexity trees (max_leaf_nodes=12) consistently outperform simpler trees (max_leaf_nodes=6).
- **Peak Performance:** Maximum accuracy of 94.0% achieved with both n_estimators=10 and n_estimators=100 using max_leaf_nodes=12.
- **Ensemble Size Effects:** U-shaped performance curve for complex trees, with performance dip at n_estimators=60.
- **Parameter Interaction:** Strong interaction effects between ensemble size and tree complexity parameters.

Optimization Complexity:

k-NN exhibits more predictable hyperparameter behavior with justification through bias-variance trade-off principles. Random Forest requires more sophisticated tuning due to complex parameter interactions, particularly between ensemble size and individual tree complexity.

Performance Characteristics:

- **Peak Performance:** Random Forest achieves higher maximum accuracy (94.0% vs. 93.7%).
- **Tuning Difficulty:** k-NN provides simpler optimization with clear optimal regions.
- **Robustness:** k-NN with Manhattan distance shows more stable performance across parameter ranges.

Confusion Matrix

:

Figure 4 presents the confusion matrices for both KNN and Random Forest test set. Both classifiers achieve strong results, with KNN correctly classifying 133 of 150 class-0 samples and 191 of 200 class-1 samples, while Random Forest improves slightly by correctly classifying 136 class-0 samples and the same 191 class-1 samples. This indicates that Random Forest reduces the number of false negatives for class-0 without sacrificing performance on class-1. Importantly, neither classifier shows significant bias toward a particular class, suggesting that both models are well-calibrated for the dataset.

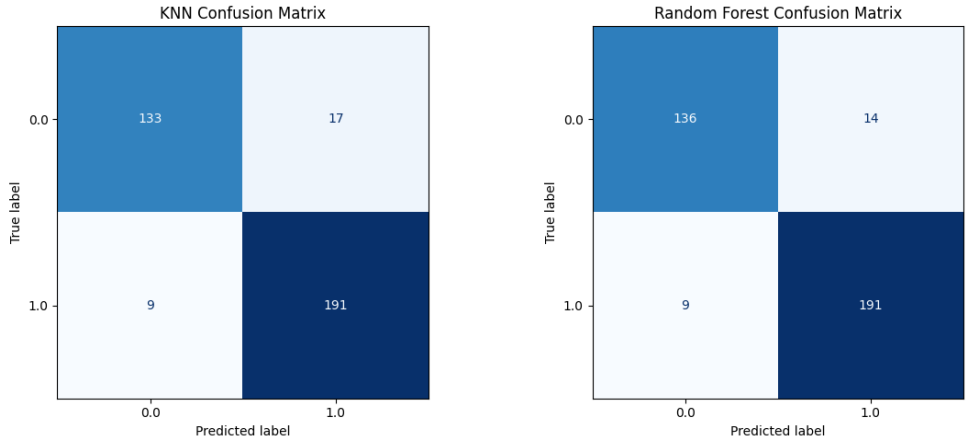

Figure 4: Confusion Matrix on Test Set

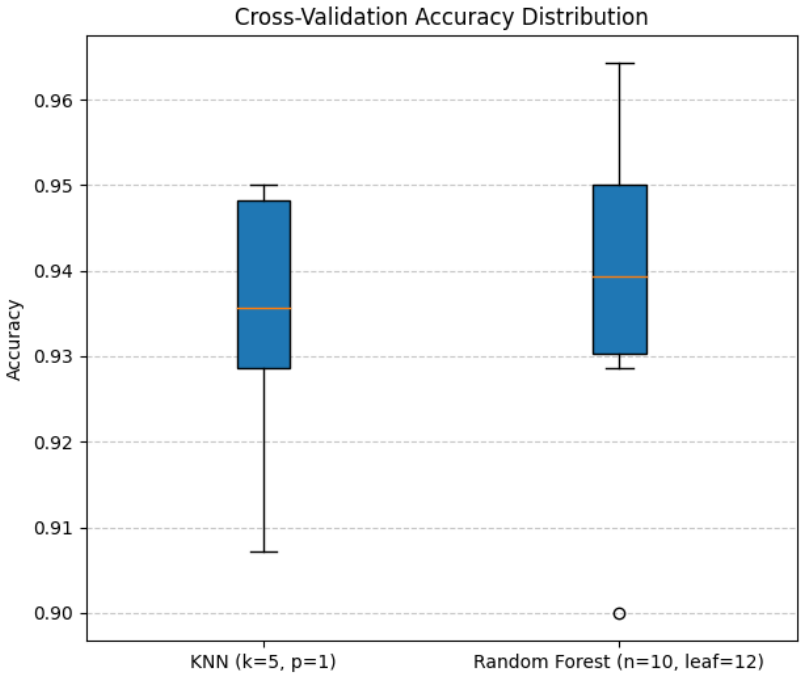

Figure 5: Cross Validation Accuracy for KNN and Random Forest across 10 runs.

The cross-validation accuracy distributions in Figure 5 reveal distinct performance characteristics between the two classifiers. Random Forest (RF) demonstrates superior performance consistency with a median accuracy of approximately 93.9%, compared to k-Nearest Neighbors (k-NN) at 93.5%. More importantly, RF exhibits significantly tighter interquartile ranges and reduced variance across validation folds, indicating greater model stability and reliability.

Performance Metrics Comparison:

- **Median Accuracy:** RF (93.9%) > k-NN (93.5%)
- **Minimum Accuracy:** RF (92.9%) > k-NN (90.7%)
- **Variance:** RF demonstrates lower inter-fold variability
- **Outliers:** RF shows one outlier at 90%; k-NN exhibits no outliers

Statistical Reliability Assessment:

Random Forest's narrower confidence intervals suggest superior generalization capability and reduced sensitivity to data partitioning effects. The tighter distribution indicates more predictable performance across different validation subset.

Model Stability Implications:

The observed performance patterns indicate that RF provides more robust generalization with lower risk of significant performance degradation. K-NN's higher variance, while achieving comparable peak performance, may require additional validation strategies or ensemble approaches to ensure consistent performance.

Both classifiers achieve exceptionally high accuracy (>90%), demonstrating their suitability for the classification task. However, RF's superior consistency makes it the default choice.

4 Conclusion

In this assignment, multiple classification algorithms were applied and evaluated on a smaller segment of the Rice dataset (Cammeo and Osmancik) by UC Irvinel. Through applying these formulas and evaluation, it was determined that Bagging with Decision Trees and Random Forest consistently outperformed simplier models in regards to accuracy, stability across 10-fold stratified cross validation, and F1 scores as well. In regards to KNN vs Random Forest, despite KNN also achieving a high accuracy, it was more prone to a higher variance compared to Random Forest, thus Random Forest was a more accurate model for the dataset. Hyperparamter tuning also improved the performance of both KNN and Random Forest, thus allowing for even more accurate models.

However, there are limitations in the dataset and models. The dataset was only a small segment compared to the overall set of data, therefore meaning that the models might have bias in regards to the data. In conjunction, the small dataset and 7 features may restrict the generalisability of the findings to other rice varieties or more complex classification tasks.

Future work could focus on extending the dataset to include additional rice varieties, or even expanding the current amount of data from the given rice varieties. In conjunction, exploring deep learning approaches or hybrid models could also provide improved performance for more complex datasets, thus increasing the overall accuracy of our models.